

Instituto Superior de Engenharia

Politécnico de Coimbra

Relatório Final do Projeto

Billing Tracker - DevOps Project

Aplicação full-stack com pipeline CI/CD, Docker, Jenkins, Docker Hub, Ansible e deploy em múltiplas VMs

Instituição	Instituto Superior de Engenharia de Coimbra
Curso	Mestrado em Engenharia Informática
Unidade Curricular	Plataformas de Desenvolvimento / DevOps
Docente	Prof. Ricardo Santos Filipe
Autores	Ruben Ferreiro; Ricardo Rodrigues
Repositório principal	https://github.com/Dragneel277/billing-pd-ruben-devops.git
Branch final	ruben-devops-fixes
Ano letivo	2024/2025

Índice

- 1. Introdução
- 2. Objetivos do projeto
- 3. Descrição da aplicação
- 4. Tecnologias utilizadas
- 5. Arquitetura geral
- 6. Arquitetura de rede
- 7. Docker e containerização
- 8. Pipeline Jenkins
- 9. Docker Hub e versionamento
- 10. Deploy automatizado com Ansible
- 11. Base de dados e persistência
- 12. Nginx e reverse proxy
- 13. Autenticação com JWT
- 14. Smoke tests e validação
- 15. Segurança e isolamento
- 16. Validação final e demonstração
- 17. Limitações e melhorias futuras
- 18. Conclusão
- Anexos

1. Introdução

O presente relatório descreve o desenvolvimento e a infraestrutura DevOps do projeto Billing Tracker, uma aplicação web full-stack para gestão de despesas e faturas pessoais. O projeto combina uma aplicação funcional com um fluxo completo de entrega automatizada, incluindo build, publicação de imagens, deployment em máquinas virtuais, validação automática e notificação por email.

A solução foi organizada para demonstrar práticas de DevOps num ambiente académico, recorrendo a Docker para empacotamento dos serviços, Jenkins para pipeline CI/CD, Docker Hub para registo de imagens, Ansible para deployment e Oracle VirtualBox para a execução das máquinas virtuais.

2. Objetivos do projeto

O objetivo principal foi implementar uma solução funcional e automatizada, capaz de transformar uma alteração no código numa nova versão da aplicação em execução nas VMs.

- Desenvolver uma aplicação full-stack para gestão de despesas e faturas.
- Containerizar frontend, backend e base de dados com Docker.
- Criar uma pipeline CI/CD com Jenkins.
- Publicar imagens Docker no Docker Hub com tags versionadas.
- Automatizar o deployment em múltiplas VMs usando Ansible.
- Separar responsabilidades entre VM da aplicação e VM da base de dados.
- Melhorar o isolamento do backend, deixando apenas o frontend exposto.
- Validar automaticamente o sistema com smoke tests.
- Garantir persistência dos dados PostgreSQL através de volume Docker.

3. Descrição da aplicação

O Billing Tracker permite que um utilizador registe e acompanhe despesas ou faturas pessoais. A aplicação inclui autenticação, operações CRUD sobre bills, filtros, categorias, estados e dashboard analítico.

Funcionalidade	Descrição
Registo e login	Criação e autenticação de utilizadores com JWT.
Gestão de bills	Criar, editar, listar e eliminar despesas/faturas.
Categorias	Classificação por rent, utilities, internet, services, food, transport e other.
Estados	pending, paid, overdue e cancelled.
Pesquisa e filtros	Pesquisa por título, entidade ou descrição, e filtros por estado/categoria/data.
Overdue automático	Bills pendentes com data vencida são apresentadas como overdue.
Analytics	Dashboard com resumo e gráficos por mês, ano, categoria e dias de maior gasto.

4. Tecnologias utilizadas

Camada	Tecnologia
Frontend	React + Vite + Nginx
Backend	Node.js + Express
Base de dados	PostgreSQL
Autenticação	JWT
Containerização	Docker
Registo de imagens	Docker Hub
CI/CD	Jenkins
Deployment	Ansible
Virtualização	Oracle VirtualBox
Ambiente local de CI	JenkinsWSL
Controlo de versões	Git + GitHub

5. Arquitetura geral

A arquitetura final segue um fluxo DevOps completo, desde o código no GitHub até à aplicação validada em execução nas VMs.

```
GitHub
  -> Jenkins Pipeline
  -> Docker Build
  -> Docker Hub
  -> Ansible Deploy
  -> Smoke Tests
  -> Email Notification
```

O Jenkins é responsável pela execução da pipeline. O Docker cria imagens para frontend e backend. O Docker Hub armazena essas imagens. O Ansible utiliza as imagens publicadas para recriar os containers nas VMs. No final, os smoke tests confirmam que a aplicação responde corretamente.

Componente	Responsabilidade
GitHub	Armazenar o código-fonte e disponibilizá-lo ao Jenkins.
Jenkins	Executar checkout, build, push, deploy, testes e notificações.
Docker	Criar imagens reproduzíveis da aplicação.
Docker Hub	Guardar imagens versionadas do frontend e backend.
Ansible	Automatizar o deployment nas VMs.
VirtualBox VMs	Executar a aplicação e a base de dados.

6. Arquitetura de rede

A infraestrutura utiliza duas VMs Ubuntu, cada uma com rede Host-only e NAT Network. A rede Host-only é usada pelo Jenkins e pelo Ansible para acesso SSH. A NAT Network é usada para comunicação interna entre a aplicação e a base de dados.

VM	Host-only IP	NAT IP	Função
billing-app-vm	192.168.56.101	10.0.2.3	Frontend + Backend
billing-db-vm	192.168.56.102	10.0.2.4	PostgreSQL

```
Jenkins/Ansible -> billing-app-vm: 192.168.56.101
Jenkins/Ansible -> billing-db-vm: 192.168.56.102
Backend -> PostgreSQL: 10.0.2.4:5432
```

Na VM da aplicação foi criada ainda uma rede Docker interna chamada billing-net, onde se ligam os containers billing-frontend e billing-backend. Esta configuração permite que o backend fique privado e seja acedido apenas pelo Nginx.

7. Docker e containerização

Docker foi utilizado para empacotar cada componente da aplicação num container isolado. Isto garante que a aplicação corre de forma consistente e facilita a automatização do deployment.

Container	Função	Exposição final
billing-frontend	React compilado servido por Nginx	0.0.0.0:80->80/tcp
billing-backend	API Node.js/Express	3000/tcp apenas interno
billing-db	PostgreSQL	Ligado à DB VM, com volume persistente
jenkins	Servidor CI/CD	localhost:8080

A imagem construída pelo Jenkins é a mesma que o Ansible executa nas VMs. Esta decisão reduz diferenças entre ambiente de build e ambiente de execução.

8. Pipeline Jenkins

A pipeline está definida em jenkins/Jenkinsfile e organiza o processo CI/CD. O Jenkins faz checkout do código, constrói as imagens, publica no Docker Hub, executa o Ansible, valida o sistema e envia notificação por email.

1. Checkout do código no GitHub.
2. Build da imagem Docker do backend.
3. Build da imagem Docker do frontend.

4. Login no Docker Hub através de credenciais do Jenkins.
5. Push das imagens com BUILD_NUMBER e latest.
6. Chamada ao ansible-playbook para deployment nas VMs.
7. Execução de smoke tests.
8. Limpeza dos dados temporários de teste.
9. Envio de email de sucesso ou falha.

```
stage('Build images')
stage('Push to Docker Hub')
stage('Deploy via Ansible to VMs')
stage('Smoke Test Multi-VM')
stage('Cleanup Smoke Test Data')
```

9. Docker Hub e versionamento de imagens

O Docker Hub funciona como registry das imagens da aplicação. Depois do build, o Jenkins publica as imagens do backend e frontend no Docker Hub.

```
dragneel277/billing-backend:<BUILD_NUMBER>
dragneel277/billing-backend:latest
dragneel277/billing-frontend:<BUILD_NUMBER>
dragneel277/billing-frontend:latest
```

A tag com o BUILD_NUMBER permite rastreabilidade, pois identifica exatamente a imagem criada por cada execução da pipeline. A tag latest aponta para a versão mais recente.

10. Deploy automatizado com Ansible

O Ansible automatiza a configuração e o deployment nas VMs. O ficheiro ansible/inventory.ini define os hosts e o ficheiro ansible/playbook.yml define as tarefas.

Ficheiro	Papel
ansible/inventory.ini	Define grupos db_servers, backend_servers e frontend_servers, com os IPs Host-only.
ansible/playbook.yml	Define as tarefas de instalação, pull de imagens, criação de rede, execução de containers e validação.

Play	O que faz
Deploy PostgreSQL database	Executa PostgreSQL na DB VM, cria volume e aplica init.sql.
Remove stale database container	Remove container antigo de DB que possa existir na app VM.
Deploy backend	Cria billing-net, puxa imagem backend e executa o container privado.
Deploy frontend	Puxa imagem frontend, executa Nginx na porta 80 e valida /api/health.

11. Base de dados e persistência

A base de dados PostgreSQL corre na billing-db-vm, no container billing-db. Para evitar perda de dados durante redeployments, é usado um volume Docker persistente chamado billing_pgdata.

```
Container: billing-db
Volume: billing_pgdata
Schema: backend/src/db/init.sql
Host de ligação do backend: 10.0.2.4
```

O schema da base de dados é aplicado pelo Ansible a partir do ficheiro backend/src/db/init.sql. Este ficheiro cria e mantém as tabelas essenciais, como users e expenses, e suporta categorias, estados e filtros.

12. Nginx e reverse proxy

O Nginx tem duas funções principais: servir o frontend React já compilado e encaminhar pedidos de API para o backend interno. O browser comunica apenas com o frontend na porta 80.

```
Browser -> http://192.168.56.101
Nginx -> /api/*
Nginx -> billing-backend:3000
Backend -> PostgreSQL 10.0.2.4:5432
```

Com esta abordagem, o backend não precisa de estar exposto diretamente ao exterior. O acesso público à API passa pelo caminho /api através do Nginx.

13. Autenticação com JWT

A aplicação usa autenticação baseada em JWT. Após login, o backend valida as credenciais, gera um token e o frontend utiliza esse token nos pedidos seguintes.

10. O utilizador envia credenciais de login.
11. O backend valida a password.
12. O backend gera um JWT.
13. O frontend guarda o token.
14. Pedidos protegidos enviam Authorization: Bearer <token>.
15. O middleware do backend valida o token antes de permitir acesso.

14. Smoke tests e validação

Após o deploy, o Jenkins executa smoke tests para validar rapidamente se a aplicação ficou funcional. Estes testes não substituem uma bateria completa de testes, mas confirmam os pontos essenciais do sistema.

Teste	Endpoint / ação
Frontend	GET http://192.168.56.101
Health da API	GET http://192.168.56.101/api/health
Registo	POST http://192.168.56.101/api/auth/register
Login	POST http://192.168.56.101/api/auth/login

Cleanup	Remoção do utilizador ci_test_<BUILD_NUMBER> na DB

O endpoint /api/health confirma que o backend está ativo e que consegue comunicar com o PostgreSQL. O registo e login validam a API, base de dados, autenticação e proxy Nginx.

15. Segurança e isolamento

A versão final da arquitetura melhorou o isolamento da aplicação. O frontend é o único serviço exposto publicamente. O backend fica privado dentro da rede Docker billing-net e a base de dados está numa VM separada.

Melhoria	Resultado
Backend sem porta pública 3000	Reduz superfície de exposição.
billing-net	Comunicação interna entre frontend e backend.
Nginx reverse proxy	API pública apenas via /api.
PostgreSQL em VM separada	Separação entre aplicação e dados.
Volume billing_pgdata	Persistência em redeployments.
Jenkins Credentials	Secrets não ficam commitados no repositório.

16. Validação final e demonstração

A validação final deve demonstrar que a aplicação e a infraestrutura estão operacionais.

- Abrir o frontend em http://192.168.56.101/login.
- Criar, editar e eliminar uma bill.
- Verificar gráficos e filtros.
- Mostrar docker ps na app VM e na DB VM.
- Confirmar que billing-backend aparece apenas como 3000/tcp.
- Executar curl http://192.168.56.101/api/health.
- Executar curl http://192.168.56.101:3000/health e confirmar que está bloqueado.
- Mostrar Jenkins Console Output com Deploy via Ansible e Smoke Test Multi-VM.
- Mostrar Docker Hub com as tags BUILD_NUMBER e latest.

17. Limitações e melhorias futuras

Apesar de funcional, o projeto foi desenvolvido num ambiente local e académico. Algumas melhorias possíveis para uma versão de produção seriam:

- Adicionar HTTPS.
- Configurar firewall/UFW nas VMs.
- Adicionar testes unitários e integração mais completos.
- Adicionar rollback automático usando tags antigas do Docker Hub.
- Usar um secrets manager externo.
- Adicionar monitorização e métricas.

- Migrar para uma infraestrutura cloud com redes privadas.

18. Conclusão

O projeto Billing Tracker demonstra uma solução full-stack funcional integrada num fluxo DevOps completo. A aplicação permite gerir despesas e faturas, enquanto a infraestrutura automatiza build, publicação, deployment e validação.

A solução final usa GitHub, Jenkins, Docker, Docker Hub, Ansible, PostgreSQL e múltiplas VMs. O frontend fica exposto na porta 80, o backend permanece privado dentro da rede Docker e a base de dados fica separada na billing-db-vm com volume persistente.

Em resumo, foi implementado um processo em que uma alteração no código pode ser construída, publicada, instalada, testada e validada automaticamente.

Anexos

A. Comandos úteis de validação

```
ansible -i ansible/inventory.ini all -m ping
curl http://192.168.56.101/api/health
curl http://192.168.56.101:3000/health
ssh ruben@192.168.56.101
ssh ruben@192.168.56.102
docker ps --format "table {{.Names}}  {{.Image}}    {{.Status}}   {{.Ports}}"
```

B. URLs finais

Componente	URL
Frontend	http://192.168.56.101/login
API	http://192.168.56.101/api
Health	http://192.168.56.101/api/health
Jenkins	http://localhost:8080